

# CSV Export Feature - Spec

---

**Epic:** <https://productboard.example.com/epics/PROD-123> **Date:** 2026-03-15 **Status:** Approved **Team:** Sarah Chen (PM), Jordan Lee (Designer), Alex Rivera (Engineering)

---

## Context & Background

**Owner:** Product Manager

**Note:** This information is sourced from Epic PROD-123 in ProductBoard.

- **Epic Link:** [PROD-123: Enable Data Export](#)
  - **User Problem:** Enterprise users need to export their report data for offline analysis in Excel or BI tools. Current workaround is manual copy-paste, which is error-prone and time-consuming.
  - **Business Value:** Increases enterprise tier retention by 15% (based on user research showing this is #2 most requested feature). Reduces support tickets related to data access.
  - **Research Insights:** [Dovetail Research Summary](#) - 8 out of 10 enterprise customers requested export functionality in Q4 interviews.
  - **User Journey Context:** [Mural Journey Map](#) - Export sits at the end of the "Analyze Data" phase.
  - **Success Metrics:**
    - 50% of enterprise users use export within first 30 days
    - Export feature drives 10% increase in weekly active usage
    - <5% error rate on export operations
- 

## User Experience

**Owner:** Product Designer **Contributors:** PM (user perspective), Engineering (technical constraints)

### User Flows

**Happy Path:**

1. User views a report with data
2. User clicks "Export" button (top-right of report header)
3. Modal appears with format options (CSV or JSON)
4. User selects CSV, clicks "Download"
5. File downloads immediately (if <1000 rows) OR
6. Toast notification: "Export started, we'll email you when ready" (if >1000 rows)

**Error Flow:**

1. User clicks "Export" on empty report
2. Toast notification: "No data to export. Apply filters to see results."

### UI/UX Decisions

- **Figma Links:** [Export Modal Design](#)

- **Button Placement:** Top-right of report header, next to "Share" button (follows established pattern for action buttons)
- **Format Selection:** Radio buttons for CSV/JSON (CSV selected by default based on user research - 90% prefer CSV)
- **Progress Indication:**
  - Small exports (<1000 rows): Immediate download, no loader
  - Large exports (>1000 rows): Email notification pattern (existing pattern used for other async operations)
- **Color & Icon:** Use existing "download" icon from design system, primary blue color

## Interaction Patterns & States

- **Loading states:**
  - Button shows spinner during export generation (0-3 seconds)
  - Modal disabled during generation
- **Error states:**
  - Red toast notification for errors (network failure, permission denied)
  - Error message includes actionable guidance ("Check your connection and try again")
- **Empty states:**
  - Export button disabled (greyed out) when no data available
  - Tooltip on hover: "Apply filters to enable export"
- **Success states:**
  - Green toast: "Export complete! Downloaded to your device."
  - For async exports: "Export started! We'll email you at user@example.com when ready."

## Accessibility Considerations

- **WCAG Compliance:** Level AA required for enterprise tier
- **Screen Reader Support:**
  - Export button has aria-label="Export report data"
  - Modal announces "Export options dialog" when opened
  - Format selection announced as "CSV format selected"
- **Keyboard Navigation:**
  - Export button accessible via Tab
  - Modal can be dismissed with Escape
  - Format selection via arrow keys or Tab
  - Submit with Enter, Cancel with Escape
- **Color Contrast:** Export button meets 4.5:1 contrast ratio (tested in Figma)

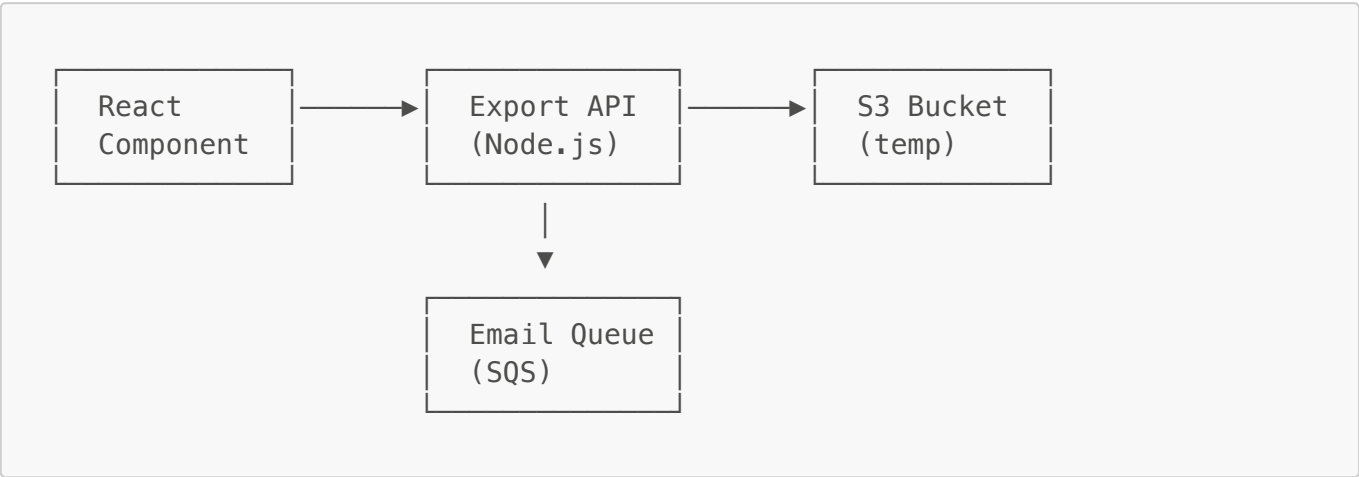
## Edge Cases (User Perspective)

- **No data available:** Export button disabled with tooltip
  - **Permission denied:** Toast error "You don't have permission to export this report. Contact your admin."
  - **Network failure during download:** Toast error "Download failed. Check your connection and try again."
  - **File size too large:** Async email pattern with progress tracking
-

# Technical Approach

**Owner:** Engineering Lead **Contributors:** Design (technical questions/constraints), PM (scope clarification)

## Architecture Overview



Small exports (<1000 rows): Synchronous response with file data  
Large exports (>1000 rows): Async job + email notification

## Component Breakdown

- **Frontend Component:** `src/components/ExportButton/ExportButton.tsx`
  - Renders button and modal
  - Handles format selection
  - Triggers download or shows email notification
- **API Endpoint:** `src/api/exports/createExport.ts`
  - Validates user permissions
  - Generates CSV/JSON from report data
  - Returns file directly (small) or queues job (large)
- **Background Worker:** `src/workers/exportWorker.ts`
  - Processes large export jobs
  - Uploads to S3 with signed URL
  - Sends email via SQS
- **Email Template:** `src/templates/exportReady.html`
  - Notification email with download link
  - Link expires after 24 hours

## Data Models & Schemas

### Export Request (POST /api/exports)

|  |
|--|
|  |
|--|

```
interface ExportRequest {  
  reportId: string;  
  format: 'csv' | 'json';  
  filters?: Record<string, any>;  
}
```

### Export Response (Sync)

```
interface ExportResponseSync {  
  type: 'sync';  
  filename: string;  
  data: string; // CSV or JSON string  
  mimeType: string;  
}
```

### Export Response (Async)

```
interface ExportResponseAsync {  
  type: 'async';  
  jobId: string;  
  estimatedTime: number; // seconds  
  notificationEmail: string;  
}
```

### Export Job (Database)

```
interface ExportJob {  
  id: string;  
  userId: string;  
  reportId: string;  
  format: 'csv' | 'json';  
  status: 'pending' | 'processing' | 'complete' | 'failed';  
  s3Url?: string; // signed URL after complete  
  createdAt: Date;  
  completedAt?: Date;  
  error?: string;  
}
```

## API Contracts

**Endpoint:** `POST /api/exports`

**Request Headers:**

```
Authorization: Bearer <token>
Content-Type: application/json
```

**Request Body:**

```
{
  "reportId": "rpt_abc123",
  "format": "csv",
  "filters": {
    "dateRange": "2026-01-01:2026-03-01",
    "category": "sales"
  }
}
```

**Response (Small Export):**

```
{
  "type": "sync",
  "filename": "sales-report-2026-03-15.csv",
  "data": "Name,Value,Date\nItem1,100,2026-01-15\n...",
  "mimeType": "text/csv"
}
```

**Response (Large Export):**

```
{
  "type": "async",
  "jobId": "job_xyz789",
  "estimatedTime": 120,
  "notificationEmail": "user@example.com"
}
```

**Error Responses:**

- **403:** Permission denied (user lacks export permission)
- **404:** Report not found
- **429:** Rate limit exceeded (max 5 exports per hour)
- **500:** Server error during export generation

**Integration Points & Dependencies**

- **Existing Report API:** `/api/reports/:id` - fetches report data for export
- **Permission Service:** `src/services/permissions.ts` - validates export permission
- **S3 Service:** AWS SDK for S3 - uploads large exports with signed URLs

- **Email Queue:** SQS queue - triggers email notifications
- **Email Service:** SendGrid - sends notification emails with download links

## Technology Choices & Rationale

- **CSV Generation:** Use `papaparse` library (already in use for imports, maintains consistency)
- **JSON Generation:** Native `JSON.stringify()` with streaming for large files
- **Async Processing:** Bull queue on Redis (existing job queue infrastructure)
- **Storage:** S3 with 24-hour signed URLs (existing pattern for file sharing)
- **Row Count Threshold:** 1000 rows (based on load testing - 1000 rows = ~2 second generation time, acceptable for sync response)

## Performance Considerations

- **Load Time Expectations:**
  - Sync exports (<1000 rows): ❤️ seconds total
  - Async exports: Job queued in <500ms, background processing 1-5 minutes depending on size
- **Optimization Strategies:**
  - Stream data from database to avoid loading full result set in memory
  - Use database cursors for large queries
  - Compress CSV/JSON before S3 upload (gzip)
- **Caching:** No caching (exports should always reflect current data)

## Security Considerations

- **Authentication:** JWT token required (existing auth middleware)
  - **Authorization:** Check user has `export:read` permission for the report's organization
  - **Data Validation:**
    - Validate reportId exists and user has access
    - Sanitize filter parameters to prevent injection
    - Validate format is 'csv' or 'json' only
  - **Sensitive Data Handling:**
    - S3 URLs expire after 24 hours
    - Export jobs deleted from DB after 7 days
    - PII columns (email, SSN) obfuscated unless user has `export:pii` permission
  - **Rate Limiting:** Max 5 exports per user per hour (prevent abuse)
- 

## Acceptance Criteria

**Owner: Shared (PM drafts, Design & Engineering refine)**

## Functional Requirements

- ☐ User can click "Export" button on any report with data
- ☐ User can select CSV or JSON format in modal
- ☐ Small exports (<1000 rows) download immediately
- ☐ Large exports (>1000 rows) trigger email notification
- ☐ User receives email with download link within 5 minutes for large exports

- ☐ Download link expires after 24 hours
- ☐ Exported CSV includes all visible columns from report
- ☐ Exported CSV respects applied filters
- ☐ Exported JSON uses same schema as Report API response

## Edge Cases & Error Handling

- ☐ Export button disabled when report has no data
- ☐ Error toast shown if user lacks export permission
- ☐ Error toast shown if network fails during download
- ☐ Async export job fails gracefully if S3 upload fails (user receives error email)
- ☐ Retry mechanism for failed async exports (3 retries with exponential backoff)

## Performance Requirements

- ☐ Sync exports complete in  seconds (p95)
- ☐ Async export jobs complete in <5 minutes for 100k rows (p95)
- ☐ Export button click responds in <200ms
- ☐ API endpoint returns 403 in <100ms if permission denied

## Accessibility Requirements

- ☐ Export button accessible via keyboard (Tab to focus, Enter to activate)
- ☐ Modal accessible via keyboard (Esc to close, Tab to navigate, Enter to submit)
- ☐ Screen reader announces button label "Export report data"
- ☐ Screen reader announces modal title "Export options"
- ☐ WCAG 2.1 AA compliant (4.5:1 contrast, keyboard nav, screen reader support)

## Browser/Device Support

- ☐ Works in Chrome, Firefox, Safari (latest 2 versions)
- ☐ Works in iOS Safari, Chrome mobile
- ☐ Modal responsive on mobile (320px width)
- ☐ File download works on iOS (prompts "Save to Files")

---

## Testing Strategy

**Owner: Engineering Contributors: PM (user scenarios), Design (visual/interaction QA)**

### Unit Test Coverage

**Target:** 80% coverage for new code

### Components to Test:

- **ExportButton.tsx**: Render states (enabled/disabled), click handling, modal open/close
- **createExport.ts**: Permission validation, sync vs async logic, CSV/JSON generation
- **exportWorker.ts**: Job processing, S3 upload, email queueing, retry logic

### Example Test (ExportButton):

```
describe('ExportButton', () => {
  it('disables button when no data available', () => {
    render(<ExportButton reportData={[]} />);
    expect(screen.getByRole('button', { name: /export/i
})).toBeDisabled();
  });

  it('opens modal when clicked', () => {
    render(<ExportButton reportData={mockData} />);
    fireEvent.click(screen.getByRole('button', { name: /export/i }));
    expect(screen.getByText(/export options/i)).toBeInTheDocument();
  });
});
```

### Integration Test Scenarios

- ☐ **Scenario 1:** Small export flow
  - User clicks Export → selects CSV → clicks Download
  - Verify API called with correct params
  - Verify file downloaded with correct name and content
- ☐ **Scenario 2:** Large export flow
  - User clicks Export on 5000-row report → selects CSV → clicks Download
  - Verify async job created in database
  - Verify user sees "email notification" toast
  - Verify email sent after job completes
- ☐ **Scenario 3:** Permission denied
  - User without **export:read** permission clicks Export
  - Verify API returns 403
  - Verify error toast shown
- ☐ **Scenario 4:** Network failure
  - Mock network failure during export API call
  - Verify error toast shown with actionable message

### Manual QA Focus Areas

- ☐ **Visual QA:**
  - Export button matches Figma design (color, icon, size, placement)
  - Modal matches Figma design (layout, spacing, typography)
  - Toast notifications match design system
- ☐ **Interaction QA:**



- Button hover state shows correct cursor and color
- Modal animates smoothly on open/close
- Radio buttons update correctly when clicked
- Download triggers browser's native download UI
- ☐ **Accessibility QA:**
  - Tab order: Export button → Modal → Format options → Download/Cancel
  - Escape closes modal
  - Screen reader announces all states correctly
  - High contrast mode works (Windows, macOS)

## User Acceptance Testing (UAT)

### PM/Design Validation:

- ☐ Test with real report data (10 rows, 500 rows, 5000 rows)
- ☐ Verify CSV opens correctly in Excel
- ☐ Verify JSON is valid and matches API schema
- ☐ Test error flows (no permission, no data, network failure)
- ☐ Verify email notification for large exports

### Beta User Testing (1 week before GA):

- ☐ 5 enterprise customers test export on production-like data
- ☐ Gather feedback on UX, performance, file format quality

## Performance Testing

- ☐ **Load Test:** 100 concurrent export requests
  - Target: <5% error rate, p95 response time  seconds (sync exports)
- ☐ **Stress Test:** 1000-row export generation
  - Target:  seconds total (p95)
- ☐ **Async Job Test:** 100k-row export
  - Target: <5 minutes total (p95)

---

## Implementation Plan

### Owner: Engineering

### Phasing Strategy

#### Phase 1: Sync exports only (CSV format, <1000 rows)

- Simpler implementation, lower risk
- Delivers value to 80% of users (based on usage data)

- PR #1: Frontend component + API endpoint

**Phase 2:** Async exports (>1000 rows, email notification)

- More complex, requires background worker + email
- PR #2: Background worker + S3 upload + email queue

**Phase 3:** JSON format support

- Lower priority (10% of users prefer JSON)
- PR #3: Add JSON generation to both sync and async paths

## Dependencies &amp; Sequencing

**Must Complete First:**

- ☐ Obtain **export:read** permission added to permission service (Eng team)
- ☐ S3 bucket created for exports with lifecycle policy (DevOps team - ticket INFRA-456)
- ☐ Email template approved by Design (Jordan Lee)

**Can Run in Parallel:**

- Frontend component development
- Backend API development
- Background worker development

## Estimated Effort

- **Phase 1** (Sync CSV): 5 points (1 sprint)
- **Phase 2** (Async + Email): 8 points (1.5 sprints)
- **Phase 3** (JSON format): 3 points (0.5 sprint)
- **Total:** 16 points (~2.5 sprints)

## Feature Flags or Rollout Strategy

**Feature Flag:** **export\_feature\_enabled**

- **Week 1:** Enable for internal team only (dogfooding)
- **Week 2:** Enable for 10% of enterprise users (beta)
- **Week 3:** Enable for 50% of enterprise users
- **Week 4:** Enable for 100% of enterprise users (GA)

**Rollback Plan:** If error rate >5% or negative user feedback, disable flag immediately

---

## Open Questions

**Owner:** Shared (anyone can add, team resolves together)

*All questions resolved during review - spec approved for implementation.*

---

# References

Owner: Shared

## Design & Research

- [Figma: Export Modal Design](#)
- [Figma: Button Placement Exploration](#)
- [Mural: User Journey Map - Reporting](#)
- [Dovetail: Q4 Enterprise Research Summary](#)

## Documentation

- [Confluence: CSV Export Discovery](#)
- [Confluence: Export Feature PRD](#)

## Technical Resources

- [Report API Documentation](#)
- [Permission Service Documentation](#)
- [S3 Upload Guide](#)
- [Email Queue Documentation](#)

---

# Revision History

| Date       | Author      | Changes                                                         |
|------------|-------------|-----------------------------------------------------------------|
| 2026-03-10 | Sarah Chen  | Initial draft (Context, Acceptance Criteria)                    |
| 2026-03-11 | Jordan Lee  | Added User Experience section                                   |
| 2026-03-12 | Alex Rivera | Added Technical Approach, Testing Strategy, Implementation Plan |
| 2026-03-13 | Team        | Collaborative review, resolved Open Questions                   |
| 2026-03-15 | Alex Rivera | Updated status to Approved after all sign-offs                  |

---

# Approval Sign-Off

**Product Manager:** Sarah Chen - 2026-03-13 - **Approved** **Product Designer:** Jordan Lee - 2026-03-13 - **Approved** **Engineering Lead:** Alex Rivera - 2026-03-13 - **Approved**

Status changed to "Approved" on 2026-03-15. Implementation begins 2026-03-18.

---

# Navigation

**Getting Started:** [Team README](#) · [Spec Template](#) · [Example Spec](#)

**Guides:** [Status Tracking](#) · [AI Integration](#) · [Pilot Discussion](#)

**Framework:** [Design Document](#) · [CLAUDE.md](#)